

“Implement auto-scaling for an AI app on Kubernetes and monitor its health and performance metrics.”

This is the **proper lab workflow** using
Docker + Kubernetes + Minikube.

Install Required Tools

Install Docker:

```
sudo apt update
sudo apt install docker.io -y
```

Start Docker:

```
sudo systemctl start docker
sudo systemctl enable docker
```

Check Docker:

```
docker --version
```

Install Kubernetes CLI:

```
sudo snap install kubectl --classic
```

Check:

```
kubectl version --client
```

Install Minikube:

```
curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-
amd64
sudo install minikube-linux-amd64 /usr/local/bin/minikube
```

Check:

```
minikube version
```

Start Kubernetes Cluster

Start Minikube:

```
minikube start --driver=docker
```

Verify cluster:

```
kubectl get nodes
```

Output should show:

Create Project Folder

```
mkdir ai-k8s-app
cd ai-k8s-app
```

Create AI Application

Create file:

```
nano app.py
```

Paste:

```
from flask import Flask, request, jsonify
app = Flask(__name__)
@app.route("/")
def home():
    return "AI Model Running in Kubernetes"
@app.route("/predict", methods=["POST"])
def predict():
    data = request.json["number"]
    result = data * 2
    return jsonify({"result": result})
if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

Create requirements file:

```
nano requirements.txt
```

```
Flask
```

Create Dockerfile

```
nano Dockerfile
```

Paste:

```
FROM python:3.9
WORKDIR /app
COPY . .
RUN pip install flask
EXPOSE 5000
CMD ["python", "app.py"]
```

Build Docker Image

```
sudo docker build -t ai-model .
```

Check image:

```
sudo docker images
```

Load Image into Minikube

```
minikube image load ai-model
```

Create Kubernetes Deployment

Create file:

```
nano deployment.yaml
```

Paste:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ai-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: ai-app
  template:
    metadata:
      labels:
        app: ai-app
    spec:
      containers:
      - name: ai-container
        image: ai-model
        imagePullPolicy: Never
        ports:
        - containerPort: 5000
        resources:
          requests:
            cpu: "100m"
          limits:
            cpu: "500m"
```

Deploy it:

```
kubectl apply -f deployment.yaml
```

Check pods:

```
kubectl get pods
```

Create Kubernetes Service

Create file:

```
nano service.yaml
```

Paste:

```
apiVersion: v1
kind: Service
metadata:
  name: ai-service
spec:
  type: NodePort
  selector:
    app: ai-app
  ports:
    - port: 5000
      targetPort: 5000
      nodePort: 30007
```

Deploy service:

```
kubectl apply -f service.yaml
```

Check:

```
kubectl get svc
```

Access Application

Run:

```
minikube service ai-service
```

Browser opens showing:

AI Model Running in Kubernetes

Enable Metrics Server

Auto-scaling requires metrics.

```
minikube addons enable metrics-server
```

Verify:

```
kubectl top pods
```

Example output:

```
ai-app-xxxxx    30m    100Mi
```

Create Horizontal Pod Autoscaler

```
kubectl autoscale deployment ai-app --cpu-percent=50 --min=2 --max=10
```

Check autoscaler:

```
kubectl get hpa
```

Example:

```
ai-app    cpu: 10%/50%    min 2    max 10
```

Generate Load

Run:

```
while true; do curl $(minikube service ai-service --url); done
```

Observe Pod Scaling

In another terminal:

```
kubectl get pods -w
```

Pods will increase:

```
ai-app-1  
ai-app-2  
ai-app-3  
ai-app-4
```

This proves **auto-scaling works**.

Monitor Health and Performance

Check CPU usage:

```
kubectl top pods
```

Check nodes:

```
kubectl top nodes
```

Check pod health:

```
kubectl describe pod <pod-name>
```

Check logs:

```
kubectl logs <pod-name>
```

Final Commands (for Assignment Output)

Run these:

```
kubectl get pods  
kubectl get svc  
kubectl get hpa  
kubectl top pods  
kubectl describe hpa ai-app
```

These show:

- AI app deployed
- Service exposed
- Metrics monitoring
- Auto-scaling implemented